

In the Tweet Sentiment Pipeline, there are different stages at which Spark performance optimizations could be applied. The Gold tweets aggregation Delta table has an execution flow that consists of a Grouping Aggregate, Shuffle, Grouping Aggregate, Shuffle, Write Into, and Result Query Stage as shown in Figure 1. This execution plan indicates that Spark performs distributed aggregations, redistributing the data across partitions before writing the results to the Delta table. In the flow, two shuffle stages are seen, which is significant given that it is a computationally expensive operation. During a shuffle operation, aggregation data is exchanged between executors via shuffle reads and shuffle writes, which increases network I/O and memory utilization. The current size of the data that is being read here is a little over 20,000 rows, which corresponds to about 834.7KB. At this size, the workload remains reasonable and does not result in shuffle spill or skew in the query profile. However, as the number of tweets ingested from the S3 Bucket grows, the possibility of a shuffle spill or skew increases, as these operations may become bottlenecks as shuffle traffic increases.

Although a severe skew is not observed in the query profile, operations such as Grouping, Aggregate, and Shuffle could cause data skew if certain grouping keys (mentions) contain more data than others. This would cause one partition to receive more records than the others, leading to straggler tasks that delay the overall stage completion and reduce parallelism efficiency for the entire cluster.

An optimization strategy to reduce the risk of skew is to repartition the dataset before aggregation to improve workload distribution across executors. Repartitioning can reduce the possibility of memory-intensive partitions while improving parallelism during the aggregation stage. Another strategy would be to enable Adaptive Query Execution, which would allow Spark to dynamically optimize execution plans at runtime.

For accounting for spills, one possible optimization strategy is to tune the Spark parameter `spark.sql.shuffle.partitions`. Tuning the shuffle partitions helps find an ideal partition size that reduces the risk of oversized partitions exhausting the executor's memory. If there are too few partitions, then they will become unreasonably large, leading to a shuffle spill. However, if there are too many partitions, they lead to unnecessary scheduling and overhead, negatively affecting overall execution performance.

In the Gold Table, a Python UDF applies the registered small-sentiment model during inference on the tweet data from the silver table. The query profile, as shown in Figure 2, indicates the runtime for this stage is about 56 minutes while processing 5MB of data. Given the data size, the execution time is high, suggesting significant computational overhead during model inference. Given that a Python UDF is being utilized, it is a likely contributor as it requires the data to be serialized and transferred between the JVM-based Spark execution engine and the Python runtime environment. Data crossing the JVM-Python boundary incurs additional serialization and deserialization costs, which can significantly increase execution latency, unlike native Spark SQL functions, which are fully optimized by Spark's Catalyst optimizer.

One potential optimization strategy would be to replace the standard Python UDF used here with a vectorized Pandas UDF. The vectorized Pandas UDFs can reduce serialization overhead when moving data between the JVM and the Python environment, thereby improving efficiency.

Figure 1 – Gold Tweets Aggregation Query Profile

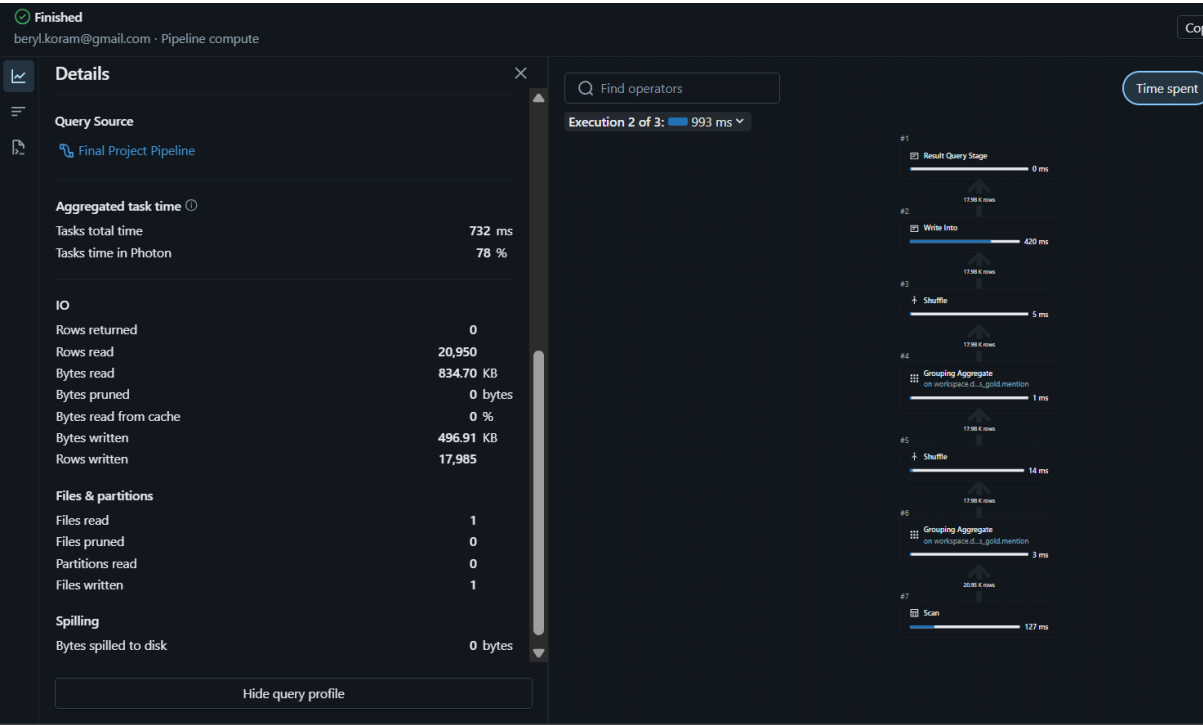


Figure 2 – Gold Tweets Table Query Profile

